

MacroAgent: Regularity-Aware Macro Legalization with LLM-Agent-Designed Contour Algorithms

JIAXI JIANG, The Chinese University of Hong Kong
 XUFENG YAO, The Chinese University of Hong Kong
 YUXUAN ZHAO, The Chinese University of Hong Kong
 YUNTAO LU, The Chinese University of Hong Kong
 PEIYU LIAO, The Chinese University of Hong Kong
 ZUODONG ZHANG, Peking University
 YIBO LIN, Peking University
 BEI YU, The Chinese University of Hong Kong

Macros constitute a large part of the core area in modern very large-scale integration (VLSI) designs. Moreover, macro positions have a significant impact on the final quality of result (QoR), and macro legalization is typically the final step in determining the macro positions. However, existing approaches related to macro legalization either lack robustness or incur substantial computational costs or neglect the regularity between macros. To address these limitations, we introduce MacroAgent. The novel framework is a four-stage approach: clustering, contour generation, template matching, and inter-cluster refinement. We propose leveraging Large Language Models (LLMs) to discover multiple, effective heuristic regularity-aware contour algorithms. This framework successfully generates robust and effective algorithmic solutions for macro legalization. Compared with state-of-the-art macro legalization works, experimental results on TILOS and Chipyard benchmarks demonstrate a 2 to 8 fold improvement in layout regularity, a 3% to 5% reduction in routed wirelength with comparable congestion after global routing, and significantly better robustness with an acceptable runtime. Furthermore, end-to-end evaluation through Cadence Innovus place-and-route confirms that the regularity improvements translate into tangible PPA gains, including 2.9% lower routed wirelength and 68.3% TNS improvement over the DREAMPlace macro legalization baseline; it also achieves 1.8% lower routed wirelength when integrated into the Innovus macro placement flow.

CCS Concepts: • **Hardware** → **Software tools for EDA**; **Methodologies for EDA**.

Additional Key Words and Phrases: Design Automation, Physical Design, Macro Legalization

ACM Reference Format:

Jiayi Jiang, Xufeng Yao, Yuxuan Zhao, Yuntao Lu, Peiyu Liao, Zuodong Zhang, Yibo Lin, and Bei Yu. 2026. MacroAgent: Regularity-Aware Macro Legalization with LLM-Agent-Designed Contour Algorithms. *ACM Trans. Des. Autom. Electron. Syst.* 1, 1 (August 2026), 27 pages. <https://doi.org/doi.org/10.1145/XXXXXXX.XXXXXX>.

1 INTRODUCTION

Macros in very large-scale integration (VLSI) circuits, such as memory and spatial arrays, are pre-designed giant functional blocks. They have fixed dimensions (e.g., height and width) and optimized structures. Macro placement heavily constrains subsequent standard-cell optimization, which in turn indirectly impacts power, performance, and area by affecting the routed wirelength [1]. Macro

Authors' addresses: Jiayi Jiang, The Chinese University of Hong Kong; Xufeng Yao, The Chinese University of Hong Kong; Yuxuan Zhao, The Chinese University of Hong Kong; Yuntao Lu, The Chinese University of Hong Kong; Peiyu Liao, The Chinese University of Hong Kong; Zuodong Zhang, Peking University; Yibo Lin, Peking University; Bei Yu, The Chinese University of Hong Kong.

2026. 1084-4309/2026/8-ART \$15.00

<https://doi.org/doi.org/10.1145/XXXXXXX.XXXXXX>

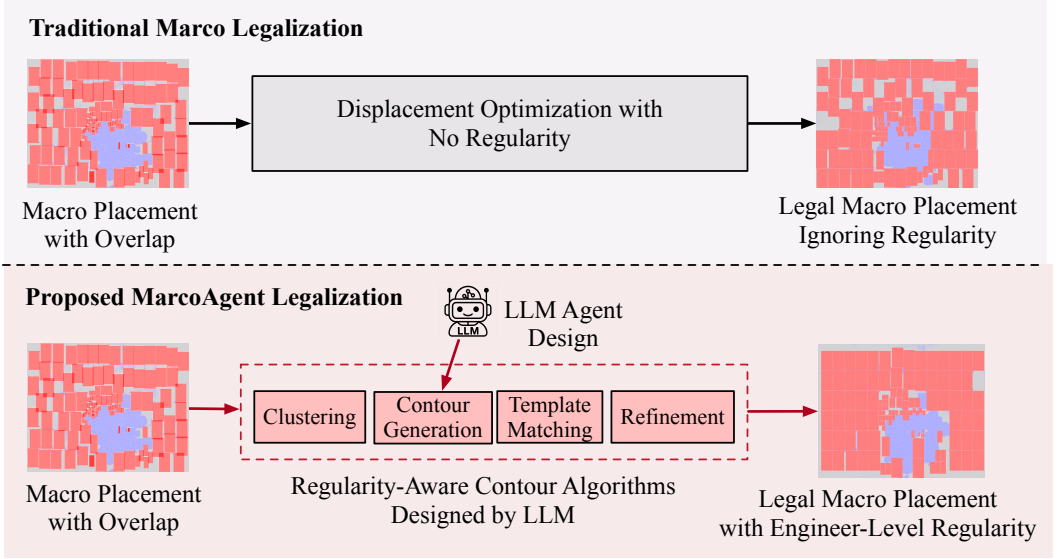


Fig. 1. The MacroAgent framework consisting of four-stage legalization steps. MacroAgent achieves engineer-level regularity macro legalization results by regularity-aware contour algorithms designed by the LLM agent.

legalization represents the final phase that determines legal macro positions at the end of macro placement process.

Traditionally, the physical design procedure for VLSI circuits is composed of multiple stages, including floorplan, macro placement, standard-cell placement, and routing. Macro placement occupies an early position in the design process and has a significant influence on subsequent phases [2]. Moreover, due to the large size of macros, minor position adjustments can significantly impact downstream optimizations [3].

The research literature categorizes macro placement into two main categories. The first is mixed-size global placement (placing macros and standard-cells together, rough locations), then macro legalization (overlap removal) [3–9]. During the first step, overlap constraints are relaxed, multiple objectives are considered. Then macro legalization removes remaining overlaps. The second category utilizes layout data structures to directly place macros, optimize customized objectives via simulated annealing [10–14]. The first category is faster than simulated annealing in the second, which is less scalable [5, 15]. In industry practice, macro placement relies on both senior engineers’ expertise (e.g., manual placement) and algorithmic efforts [5]. Complex projects may take hours to days [5]. The industrial tool, Innovus, offers a flow similar to the first macro placement method in the research literature, with two commands: `place_design -concurrent_macro` (mixed-size global placement) and then `refine_macro_place` (macro legalization) [15].

Current Approaches to Macro Legalization. Most prior works in macro legalization only attempt to minimize the displacement (i.e., the movements of macros). [16] proposes a legalization scheme that computes macro coordinates through the iterative adjustment of two constraint graphs, minimizing displacement by solving the linear programming problem. [4] extracts search points and feasible regions using the Puzzle algorithm, and iteratively places macros. DREAMPlace 2.0 [17] combines the two techniques mentioned above and has been applied in [3, 5, 9]. The work in [6] designed an occupancy-aware macro legalization algorithm that iteratively eliminates overlaps via heuristic approaches. The study in [7] employs integer linear programming or simulated annealing

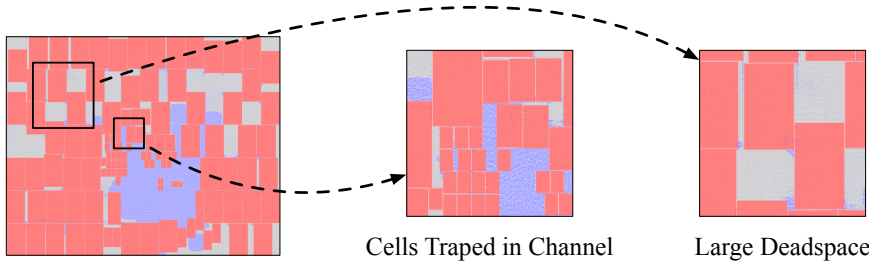


Fig. 2. Consequences of ignoring regularity in macro legalization (red giant rectangles are macros, blue tiny points are cells). Both situations worsen the wirelength.

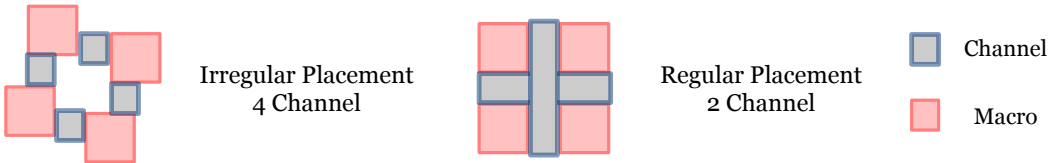


Fig. 3. Channel-based macro regularity examples.

to adjust macro positional relationships when initial sequence pairs prove infeasible. Although these techniques can improve the robustness of macro legalization, they still fall short in two aspects: **first, they overlook the regularity between macros; second, they lack robustness since only one or two heuristics cannot fit all different testcases in macro legalization.**

Regularity-Aware Macro Placement. Among prior works on macro placement, there is a common focus on macro regularity. *Studies have confirmed that such regularity reduces wirelength and minimizes deadspace [4], although the definition of regularity in these studies is quite different.* [11] uses cost models for macro grouping and regularity penalties, demonstrating a significant reduction in wirelength. [13, 14] exploit RTL hierarchy information to cluster macros and arrange them in regular patterns. [3] defined macro regularity as the necessity for macros to be surrounded by other macros or die boundaries, identify irregular placements and adjust. The MaskRegulate [8] method implements a reinforcement learning policy as a regulator to adjust existing layouts. [4] extract macros of the same type and similar levels, arrange them in array form. They consider regularity during refinement or directly place macros. **However, no academic work directly considers regularity in the final legalization stage in macro placement.** Only industrial tool Innovus provides an option `place_global_align_macro` to achieve regular macro legalization [15]. It is important to differentiate our legalization-focused approach from constructive regular placement methods [4, 8, 11, 13, 14]. While constructive methods enforce patterns early, modern mixed-size global placement tends to disrupt these pre-defined patterns during optimization, so they need to keep the macros fixed. Therefore, we do not aim to compete with placement algorithms in generating the initial structure; rather, we propose an essential complementary capability: recovering and enforcing regularity during the final legalization stage, similar to the `place_global_align_macro` flow in Innovus [15].

Regularity Metrics. Although regularity is essential for macro placement, there is no universally accepted definition. Earlier studies by [4, 13, 14] mainly characterize regularity via array-based placement configurations. [3, 8, 11] define regularity by the proximity of macros to chip boundaries or the presence of other macros in specific vicinities.

We generally adhere to the perspective in [14] that **mandating macro placement at the layout periphery is no longer feasible, as this increases the required stack depth of macros**. However, in [4, 13, 14], they constrain macros to a strictly array-based form, with no metric to evaluate their regularity.

We introduce a quantitative macro regularity metric by counting channels between macros, aligned with previous studies [4, 13, 14]. This metric reflects these physical design principles: excessive channels increase deadspace and restrict standard-cell placement optimization. As shown in Fig. 2, the cells trapped in the channels between macros are less likely to jump out of the channels, due to the density constraint [3]. It is also evident that the deadspaces between macros increase the wirelength. Our approach assigns peak regularity scores to squares and high scores to rectangles, supporting established array-based placement methods [4, 13, 14]. Fig. 3 indicates the number of channels under irregular and regular placement, which reveals that the number of channels is the smallest in the case of array-based placement. For normalization, after counting the channels, we divide the theoretical smallest possible number of channels by the acquired number. The number is less than 1, where the closer the index value is to 1, the higher the regularity achieved. We achieve a theoretical minimum number of channels when arranging macros in a square array. The following formula can obtain a rough estimation:

$$a = \lfloor \sqrt{N_{\text{macro}}} \rfloor, \quad C_{\min} = 2a - 2, \quad (1)$$

where N_{macro} is the number of macros, a is the side length lower bound of the square array, and $C_{\min}$ is the theoretical minimum number of channels. Intuitively, we need at least $a - 1$ channels to separate the macros into a rows and another $a - 1$ channels to separate the macros into a columns. Any irregular placement introduces extra channels. Finally, the regularity index is defined as:

$$R = \frac{C_{\min}}{C_{\text{actual}}}, \quad (2)$$

where C_{actual} is the actual number of channels. To compute C_{actual} , we construct a Hanan grid from all macro vertex coordinates, mark occupied cells, and iteratively merge adjacent empty cells into maximal rectangular strips. An empty cell at a crossing point may participate in merges in both directions simultaneously. The number of resulting strips yields C_{actual} ; details are provided in Section 2. Fig. 3 also exhibits that the theoretical minimum number of channels is 2. The regularity indices are 0.5 and 1 for the left and right cases, respectively.

Large Language Models for Algorithm Design. Large Language Models (LLMs) have drawn much research attention for their great performance in various cognitive tasks [18–23]. In Electronic Design Automation (EDA), more researchers are using LLMs as research tools [24–26], in script/RTL generation, circuit design, and QA systems.

Traditional automated algorithm design methods, such as genetic programming (GP) [27], require defining a set of allowed mutation operations (or primitives). Designing such a suitable set of primitives is non-trivial and remains an open challenge in practice [28]. Reinforcement learning (RL) has also been applied to algorithm design, including program superoptimization [29] and discovering faster assembly-level sorting algorithms [30]. However, RL-based approaches require human experts to carefully design the action spaces, and this design becomes difficult for higher-level programming languages [23]. Moreover, RL models are trained on specific tasks and do not readily generalize to other problems. In contrast, LLMs have been trained on vast amounts of code and have learned common patterns and routines from human-designed programs. By leveraging this knowledge together with prompt-provided context, LLMs can generate more effective suggestions than the random mutations typically used in GP [23]. Furthermore, LLMs do not require a predefined action space or mutation operations, offering greater flexibility to explore a larger search space.

Recently, there is growing interest in LLM-based algorithm design [31–34]. LLMs play various roles, such as evaluators and optimization engines. The most promising is using LLMs to directly design algorithm components (heuristic methods) for NP-hard problems [31]. In EDA, the direct use of LLMs for algorithm design is underexplored; only [35] has attempted to enhance global placement algorithms via iterative LLM implementation, using crafted prompts and an LLM-based genetic process, but lacking meaningful feedback (only wirelength). AlphaEvolve [36] combines LLMs with evolutionary search and achieves impressive results in mathematics and algorithm optimization, where LLMs already possess sufficient knowledge from pretraining. However, applying LLMs to specialized fields like EDA, where they lack domain expertise, presents a distinct challenge. SATLUTION [37] targets NP-complete SAT problems and demonstrates that LLMs can handle substantial code generation at scale. Nevertheless, as the authors discuss in the appendix, human guidance is still incorporated to manually direct higher-level algorithmic strategies while leaving the lower-level implementation to the LLM agent. GPU Kernel Scientist [38] tunes existing GPU optimization techniques (e.g., increasing thread block occupancy and resolving shared memory conflicts) to find the best kernel for specific hardware, rather than discovering new algorithms. Our work differs from these approaches in three key aspects: (1) we introduce a **domain-agnostic abstraction** that reformulates the EDA-specific problem into a geometric problem. Domain-specific decisions such as clustering strategy and optimization objective selection are encoded in the framework by human engineers, while the LLM operates solely on the abstracted geometric subproblem using a general-purpose, off-the-shelf model without any domain-specific fine-tuning. (2) Our LLM autonomously discovers both algorithm ideas and implementations **without human strategy guidance** in the intermediate process. (3) The manageable scale of macro legalization instances enables us to provide human-crafted solution samples and per-iteration visual feedback to the LLM, allowing it to understand **why** a solution is good or bad rather than relying solely on scalar metrics.

Despite their promising potential, significant limitations persist in employing LLMs for advanced algorithm design: they struggle with complex algorithm implementations and lack domain-specific expertise for specialized optimization tasks [32].

Motivation. In the EDA domain, the majority of problems are NP-hard. Researchers have historically developed numerous heuristic algorithms to address these challenges. Specifically within macro legalization, previous researchers have designed sophisticated heuristics for macro clustering, placement selection, and perturbation techniques to optimize solution quality. It naturally raises a fundamental question: Can LLMs effectively replace human experts in designing heuristic algorithms for such problems?

Our Contribution. To address these challenges, we propose a robust regularity-aware macro legalization framework with four stages. Our framework decomposes the macro legalization process into four sequential components of clustering, regularity-aware contour generation, template-based macro matching, and inter-cluster refinement as shown in Fig. 1. **Compared with the traditional legalization methods, our methods are more robust through the combination of multiple heuristic LLM-designed contour algorithms. It achieves simultaneous regular macro legalization while minimizing the displacement through template matching.** To mitigate the limitations of LLMs in EDA domains, we extract a domain-agnostic geometry problem and present LLMs with the engineers’ manual macro legalization solutions, which address gaps in domain knowledge and simplify the original problem. This enables fast, automatic, and diverse heuristics findings in an offline manner to fit different input testcases (no human intervention in the intermediate process). Notably, the hard testcases used for final validation are distinct from those

utilized to guide LLMs in heuristic discovery, effectively demonstrating the strong generalization capability of LLM generated heuristics. We validate MacroAgent under two experimental flows. In the academic flow (DREAMPlace + HeLEM-GR), MacroAgent reduces routed wirelength by 3%–5% with comparable congestion on both TILOS and Chipyard benchmarks. In the industrial flow (Cadence Innovus place-and-route), MacroAgent achieves 2.9% lower routed wirelength, 68.3% TNS improvement, and comparable power over the DREAMPlace macro legalization baseline; it also achieves 1.8% lower routed wirelength when integrated into the Innovus macro placement flow.

We organize the rest of the paper as follows: Section 2 introduces the rise of LLM agents in algorithm design and the problem definition of macro legalization. Section 3 introduces the proposed macro legalization framework. Section 4 introduces the implementation details of the proposed macro legalization framework. Section 5 presents the experimental results. Section 6 concludes the paper.

2 PRELIMINARIES

2.1 LLM Agents for Algorithm Design

An LLM agent is an AI that uses LLMs to autonomously perform tasks by reasoning, taking step-by-step actions, and leveraging tools. Currently, LLM agents are revolutionizing algorithm design through automated code generation and optimization. Recent studies [32] identify several distinct roles that LLMs play in algorithm design: (1) **LLM as optimizer**: OPRO [33] uses LLMs to directly generate and refine solutions through prompt-based optimization, treating the LLM itself as the search operator; (2) **LLM as evaluator**: Jawahar et al. [34] leverage LLM performance predictors as initializers for neural architecture search, where the LLM assesses candidate quality rather than generating algorithms; (3) **LLM as heuristic generator**: this is the most promising paradigm for NP-hard problems, where LLMs generate complete algorithm code within an evolutionary framework. Google’s FunSearch [23] and AlphaEvolve [36] exemplify this third paradigm, demonstrating AI’s capacity to discover and refine algorithms across scientific and engineering domains. EoH [31] further advances this direction by evolving both algorithmic ideas and their implementations simultaneously. The core idea is evolutionary code generation and evaluation. LLM agents autonomously generate, modify, and evolve complete algorithm code over multiple iterations. And evaluation systems guide the algorithmic evolution process toward better solutions. This methodology essentially creates a self-improving cycle to continuously refine results. Traditional automated algorithm design methods rely on genetic programming or reinforcement learning, both of which require experts to design the mutation operations or action spaces. In contrast, LLM agents directly generate algorithms within the algorithm space using high-level programming languages. In EDA, EvoPlace [35] applies this paradigm to global placement by iteratively evolving placement heuristics via LLM-based code generation.

2.2 Macro Legalization Problem Formulation

Problem 1. Given macro positions after mixed-size placement $M = \{m_1, m_2, \dots, m_n\}$, where each macro m_i has (1) initial coordinates (x'_i, y'_i) (representing bottom-left corner), (2) fixed dimensions: width w_i and height h_i . Find legal macro positions $\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ that:

- (1) **No Overlap**: For any two distinct macros $m_i, m_j \in M$, there is no overlap between them, as depicted in Equation (3).

$$[x_i, x_i + w_i] \times [y_i, y_i + h_i] \cap [x_j, x_j + w_j] \times [y_j, y_j + h_j] = \emptyset, \quad (3)$$

- (2) **Minimum Displacement:** Minimize the total Manhattan displacement of all macros. The displacement of m_i is $|x'_i - x_i| + |y'_i - y_i|$, so the displacement objective is

$$\min \sum_{i=1}^n (|x'_i - x_i| + |y'_i - y_i|), \quad (4)$$

- (3) **Maximum Regularity:** Regularity index R is quantified by the channel ratio metric. For a macro placement, R is defined as:

$$R = \frac{C_{\min}}{C_{\text{actual}}}, \quad (5)$$

where $C_{\min}$ refers to the theoretical minimum number of channels and C_{actual} is the actual number of channels.

Channel Counting Method: To compute C_{actual} in practice, we employ a Hanan-grid-based procedure. First, we collect all x - and y -coordinates of every macro vertex and construct a Hanan grid, i.e., a rectilinear grid whose lines pass through every macro boundary. Each grid cell occupied by a macro is marked as **occupied**; the remaining cells are **empty**. We then iteratively merge adjacent empty cells into maximal rectangular strips: vertically adjacent empty cells sharing the same width are merged, and horizontally adjacent empty cells sharing the same height are merged. An empty cell at a channel crossing point may participate in merges in both directions simultaneously (i.e., it is shared by multiple strips rather than consumed by one). The merging repeats until no further merges are possible, and the number of resulting merged strips equals C_{actual} . This method naturally handles general layouts: more irregular placements produce more fragmented strips, yielding higher C_{actual} and thus lower regularity index R .

3 ALGORITHM DESIGN

3.1 Overview of MacroAgent

Our proposed macro legalization consists of four principal components: size-aware distance-based clustering, regularity-aware contour generation, template-based macro matching, and inter-cluster refinement. We identify regularity-aware contour generation as the most influential component, as it directly enables macros within a cluster to be arranged in a highly regular configuration. We employ an LLM agent to optimize this critical component. This choice follows the LLM-based algorithm discovery paradigm [23, 36], which requires an automated evaluator for the iterative generate-evaluate loop. Matching already admits an optimal solution via the Hungarian algorithm, leaving no room for improvement. Clustering lacks a direct quality metric: its output can only be assessed after running the downstream pipeline, whose key component (contour generation) is itself heuristic. This makes the end-to-end result quality unreliable and creates a credit assignment problem that hinders LLM optimization. In contrast, contour generation’s immediate downstream is matching, which is optimally solved by the Hungarian algorithm; therefore, the result quality of a contour algorithm can be evaluated directly and reliably via regularity and displacement metrics.

Overview. *The cluster step determines which macros are suitable for regular placement. The use of regularity-aware contour generation helps achieve regularity and diverse regular patterns for robustness; template-based macro matching can minimize displacements; and inter-cluster refinement further ensures no overlap — all of these techniques working together to generate high-quality macro legalization results.*

3.2 Size-Aware Distance-Based Clustering

Consistent with prior research, we first cluster macros. Previous approaches usually determine clustering by macro dimensions, interconnection relationships, positions in the placement prototype, or hierarchical associations [4, 11, 13, 14]. Our clustering method simplifies this by considering only macro dimensions and positions in the placement prototype, as interconnection relationships were partly accounted for in the mixed-size global placement stage. Since many similar heuristic algorithms have been proposed before, we utilize established methods instead of LLM-based approaches for this algorithm design.

Algorithm 1 Size-Aware Distance-Based Macro Clustering.

Input: Macro set $\mathcal{M}$, size grouping factor α , proximity factor β

Output: Assignment of macros to clusters

```

1:  $clusterID \leftarrow 0$ ;
2: Size grouping:  $groups \leftarrow$  group macros by size using  $\alpha$ ;
3: for each size group  $g \in groups$  do
4:   construct KD-tree from macro center points;
5:   for each unvisited macro  $m \in g$  do
6:      $current \leftarrow clusterID$  then increment  $clusterID$ ;
7:     BFS traversal of neighboring macros;
8:     For current macro  $m_c$ , calculate  $T_x = w(m_c) \cdot \beta$ ,  $T_y = h(m_c) \cdot \beta$ ;
9:     Find all macros within horizontal distance  $T_x$  and vertical distance  $T_y$ ;
10:    Assign them to  $current$  and continue traversal;
11:   end for
12: end for
```

Algorithm 1 comprises two procedures: dimensional classification and spatial clustering. The algorithm first utilizes parameter α to partition macros based on dimensional similarity. Dividing the macro width and height by α , yields dimension identifiers, which are then used to group macros with close dimensions. Subsequently, within each dimensional group, the algorithm employs parameter β in conjunction with a KD-tree to perform proximity-based clustering. In our experiments, we fix $\alpha = 10$ and $\beta = 1.5$. Specifically, $\alpha = 10$ means that we treat macros' size difference less than 10 site width as the same size (as DREAMPlace scales site width to 1). $\beta = 1.5$ means that we consider macros with a distance less than 1.5 times their respective sizes as neighboring macros. In pilot trials on the easy cases, we observed that modest variations around these values only lead to slight changes in cluster granularity. Notably, we implement adaptive proximity thresholds (T_x and T_y) that are proportional to macro's dimension sizes, enabling larger macros to have expanded search domains. In comparison, smaller macros maintain narrower search scopes. This clustering efficiently aggregates macros with similar dimensions and spatial proximity into clusters. Note that α serves solely as a grouping granularity for this clustering stage; once clusters are formed, all subsequent stages—contour generation (Section 3.3), template generation, and macro matching—operate on the original, unscaled macro dimensions. Likewise, the regularity metric R in Equation (5) and channel counting are evaluated using the actual macro sizes and legalized positions.

3.3 Regularity-Aware Contour Generation

Regularity-aware contour generation is the key step to achieve high regularity while minimizing displacement. Previous research tends to place the clustered macros in a rectangular formation [4, 13, 14]. In other words, the contour of the macro cluster is a rectangle, as it is the simplest regular shape.

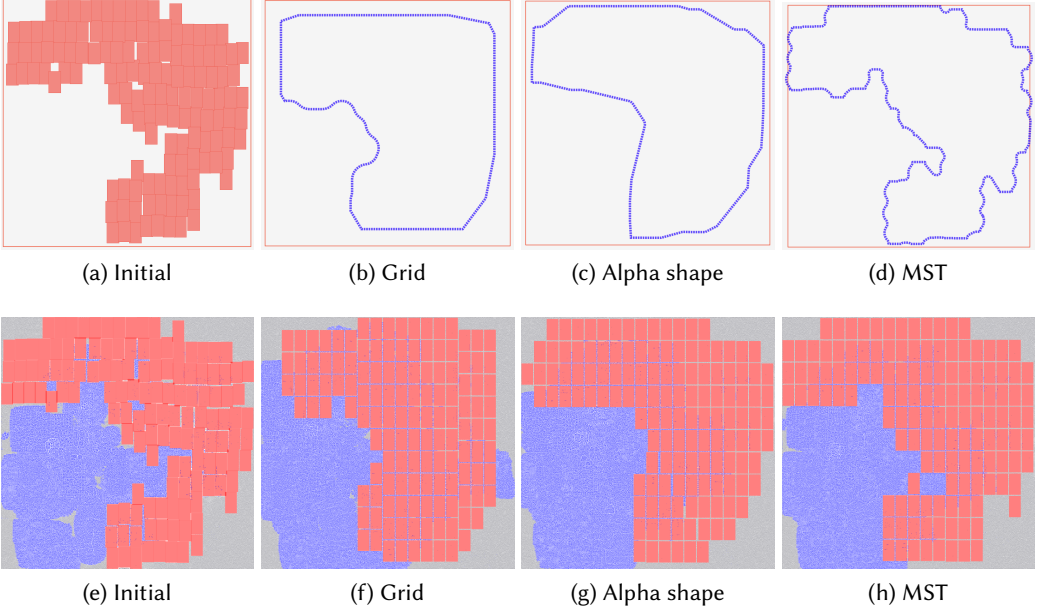


Fig. 4. Visualizations of different contour algorithms in *Ariane133*: (top row) contour visualization and (bottom row) layout after standard-cell placement (red for macros; blue for standard cells). From left to right, regularity decreases, but displacement improves. Initial is for original global placement outputs, and Grid, Alpha shape, and MST are for different legalization results.

However, our empirical observations reveal that many macro clusters after mixed-size placement exhibit contours that deviate significantly from rectangular shapes as shown in top-left figure in Fig. 4. Enforcing rectangular arrangements of macro clusters often results in displacement-induced degradation of design quality. Consequently, we propose generating diverse regularity-aware contours based on the outline morphology of macro clusters after mixed-size placement. The regularity part of contour algorithms lies in the removal of some burrs on the contour, as shown in Fig. 4. The fewer burrs, the better the regularity, but the worse the displacement. **We compare the manual designed rectangle contour heuristic with LLM-designed contour generation heuristics in the experiments Fig. 13 to prove that LLM-designed methods can achieve better design quality.**

Necessity of Multiple Contours.. The reason for diverse contours is that regularity and displacement are conflicting, so we need to trade them off; furthermore, there are no direct analytical solutions to guide this trade-off (only evident after standard-cell placement). We here present the results of *Ariane133* under different contours. Fig. 4 shows the three macro legalization results generated for *Ariane133*. The picture on the far left is the result of an initial mixed-size placement. The three pictures on the right are the results generated by three different contour algorithms, respectively. Smoothing contour burrs improves the channel metric (higher regularity) but typically increases macro displacement; different contours (Grid, Alpha-shape, MST) realize different points on this trade-off frontier.

LLM Agent-Based Contour Algorithm Generation Flow. Fig. 5 demonstrates the end-to-end workflow for discovering contour-generation algorithms with the proposed LLM agent. Prompts and

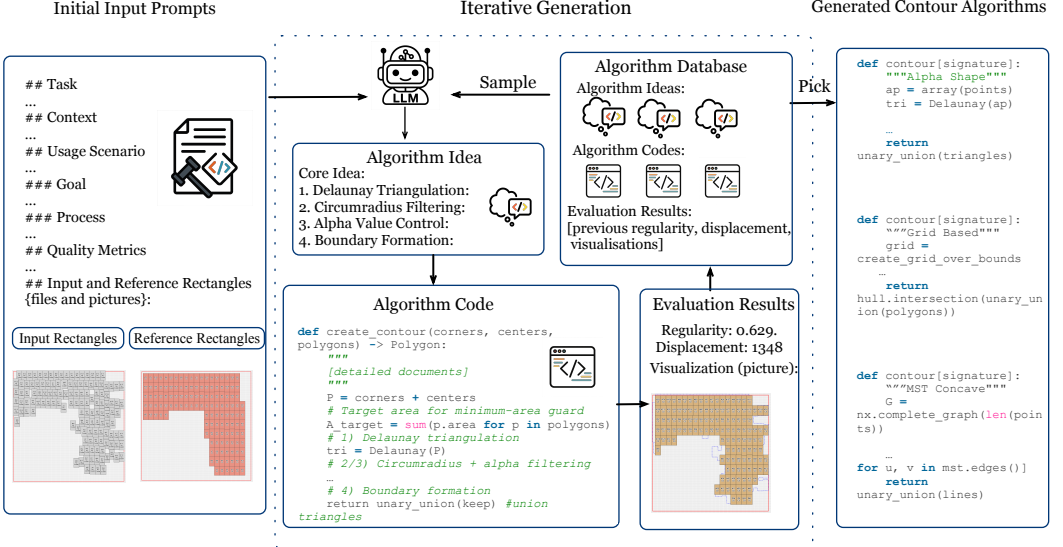


Fig. 5. LLM agent-based contour algorithm generation flow. Key components: (1) **Input Prompt:** Contains task description, metrics (regularity, displacement), previous ideas/code, evolution directive (E1/E2/M1/M2/M3), and reference solutions; (2) **LLM Agent:** Processes the prompt and generates new algorithm ideas and executable Python code; (3) **Code Execution:** Runs the generated contour algorithm on test cases; (4) **Evaluation:** Computes regularity and displacement metrics, generates visualizations; (5) **Algorithm Database:** Stores successful algorithms that outperform existing ones on any test case; (6) **Feedback Loop:** Results feed back into the next iteration’s prompt to guide further evolution.

code are simplified due to page size limits. Initially, we craft a comprehensive prompt that specifies (1) task, (2) usage scenario, (3) quality metrics (regularity, displacement), (4) prior ideas/code, (5) evolution directive (one of E1/E2/M1/M2/M3), (6) outcomes, and (7) reference manual legalization solutions. The content of the simplified version of the “initial input prompt” is shown in Fig. 6. At the first initialize round, the “previous idea” is the simple **rectangle contour** generation algorithm: it computes the bounding box of the macro cluster, scales the rectangle to match the cluster’s area, and recenters it on the cluster.

Specifically, the workflow in Fig. 5 comprises six key components: (1) **Input Prompt** encapsulates the complete problem specification fed to the LLM, including the task description, evaluation metrics, prior algorithm ideas and code, an evolution directive, and reference solutions (detailed below with Fig. 6); (2) **LLM Agent** receives the assembled prompt and generates a new algorithm: a natural-language *idea* describing the heuristic logic, together with an executable Python *code* implementation of the contour generation function; (3) **Code Execution** runs the generated contour algorithm on the macro cluster testcases, producing contour polygons that are subsequently used for template generation and macro matching; (4) **Evaluation** computes the regularity metric R and displacement for each testcase and generates layout visualizations, enabling quantitative comparison against existing algorithms; (5) **Algorithm Database** archives every algorithm whose regularity or displacement outperforms all existing entries on at least one testcase, forming a growing Pareto-front repository; (6) **Feedback Loop** feeds the evaluation results and the current algorithm database back into the next iteration’s prompt, allowing the LLM to learn from prior successes and failures and progressively refine its designs.

```

Initial Input Prompts

## Task
Write a Python class in `{file}.py` named `ConcaveHullGenerator`. Implement this method
using shapely without changing the signature, specifically treating corners and centers as
lists of (x,y) coordinates and polygons as Shapely objects, to generate a robust enclosing
contour...

## Context
- **Previous method ideas**: {ideas}
- **Previous results**: {displacement}, {regularity_index}, {solutions} and {visualization}
- **Evolution direct**: [E1/E2/M1/M2/M3][detailed instructions about evolution directions]

## Usage Scenario
[input descriptions]

### Goal
[detailed class requirements ]

### Process
[detailed procedures that how this class is used]

## Quality Metrics
[displacement and similarity]

## Input and Reference Rectangles {files and pictures}:
...

```

Fig. 6. Simplified version of the “initial input prompt” for LLM agent-based contour algorithm generation. Main sections: (1) **Task Description**: Defines the contour generation problem and its role in macro legalization; (2) **Metrics**: Specifies regularity (channel count) and displacement objectives; (3) **Previous Ideas**: Provides natural-language descriptions of prior algorithms (e.g., rectangle, convex hull); (4) **Previous Code**: Shows executable Python implementations of prior algorithms; (5) **Evolution Strategy**: Instructs the LLM on how to modify/improve (E1: explore diversity, E2: synthesize and innovate, M1: fix weaknesses, M2: tune parameters, M3: simplify); (6) **Reference Solutions**: Includes manual legalization examples to guide the LLM toward practical solutions.

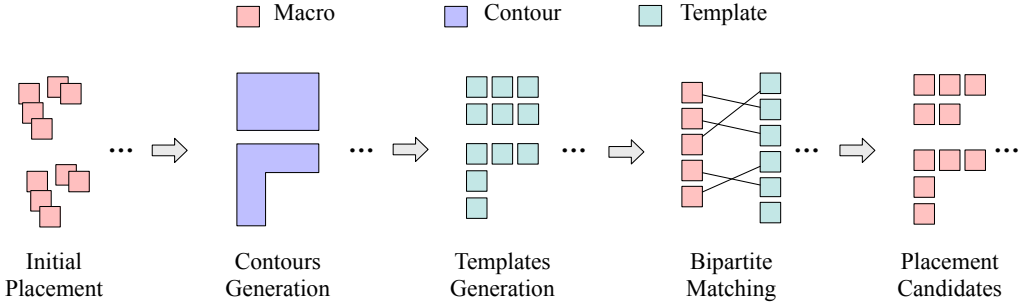


Fig. 7. Template-based macro matching. The process consists of three stages: (1) a regularity-aware contour is generated around the macro cluster; (2) a uniform grid template is tiled inside the contour and feasible grid cells are selected; (3) macros are assigned to grid positions via the Hungarian algorithm to minimize total displacement.

Fig. 6 shows a simplified version of the input prompt, which is organized into six sections: (1) **Task Description** defines the contour generation problem: given a set of macro positions and dimensions within a cluster, produce a closed rectilinear polygon that encloses the macros with appropriate regularity. (2) **Metrics** specifies the two optimization objectives: regularity (measured by the channel count ratio R) and displacement (Manhattan distance from original positions), establishing the evaluation criteria the LLM must optimize. (3) **Previous Ideas** provides natural-language descriptions of prior contour algorithms (e.g., rectangle bounding box, convex hull, alpha shape), giving the LLM conceptual building blocks to draw upon or diverge from. (4) **Previous Code** includes the executable Python implementations corresponding to the previous ideas, enabling the LLM to understand concrete implementation patterns and reuse effective code structures.

(5) **Evolution Strategy** instructs the LLM on *how* to modify or improve the previous algorithms using one of five directives adopted from [31]. Two are exploration strategies: E1 selects p parent heuristics and asks the LLM to generate a maximally different new heuristic; E2 first summarizes the common ideas behind p parents and then creates a new heuristic that builds on those shared insights yet diverges substantially. Three are modification strategies: M1 selects a single parent and asks the LLM to diagnose its weaknesses and revise both idea and code; M2 adjusts only the parameters of a single parent while preserving its algorithmic structure; M3 identifies and removes redundant components to simplify the implementation. (6) **Reference Solutions** includes manually crafted macro legalization examples for the testcases, providing the LLM with concrete targets that demonstrate what high-quality placements look like and guiding it toward practically effective contour shapes.

Then, we design a generation flow that evolves both *ideas* and *code* via the agent flow adopted from [31]. The process jointly optimizes **ideas** (natural-language heuristic logic) and **code** (executable Python functions) under an **initialize** $\rightarrow$ **generate** $\rightarrow$ **evaluate** $\rightarrow$ **iterate** framework: (1) **Initialize**: Create N initial heuristics via the initial input prompt; (2) **Generate**: Until the **G-generation** stopping criterion, we apply **5 prompt strategies** to produce new candidates per generation. Previous ideas are sampled probabilistically (sample number $p=5$ for E1/E2; $p=1$ for M1–M3). Prompts combine previous *ideas* and *code* to produce varied outputs across conceptual and implementation domains; (3) **Evaluate**: Evaluate on problem testcases; admit only feasible solutions to the algorithm database; (4) **Iterate**: Sample the p candidates from the algorithm database to form the next generation; repeat until completion, then output the best heuristics. We propose five prompt strategies to improve the performance: (1) **E1 (exploration)**: Generate heuristics maximally different from the five samples to expand diversity; (2) **E2 (exploration with synthesis)**: First summarize common ideas across the five samples, then propose substantially different heuristics to ensure effective innovation; (3) **M1 (structural modification)**: Diagnose a single sample’s shortcomings and revise both idea and code to improve performance; (4) **M2 (parametric tuning)**: Adjust parameters only, preserving the algorithmic structure; (5) **M3 (simplification)**: Remove redundant code components to improve efficiency.

To control cost and ensure reproducibility, we adopt a fixed budget: $N = 10$ initial heuristics and $G = 20$ generations. These hyperparameters are aligned with the prior work [31]. The only difference is that here, to control the cost, we only use each prompt strategy once instead of N times in [31]. In total, we generate $N + G \times 5 = 110$ heuristics.

We modify the population management strategy in the prior work [31]. Instead of using a fixed size population, we decide to keep the heuristics if they have better regularity or displacement than existing ones on any testcases.

We add extra prompts before algorithm idea/code generation. These prompts make the LLM output structured contents we need. Usually, the LLM follows the structures (function signature) mentioned in the prompts. If the LLM contradicts the structure we need, we directly discard its output and start next iteration. Since this rarely happens, this method does not hinder the process.

3.4 Template-Based Macro Matching

After generating regularity-aware contours, we create templates based on them. Here, a template refers to a grid structure where each cell in the grid can accommodate a macro. Utilizing the template naturally leads to regular macro legalization, and our objective is to minimize the displacement. Matching is a common technique; we adopt a similar template-based approach as in [39]. Each cluster is matched separately, and the influence between different clusters is not considered in this step.

Contour-based Template Generation. The template grids are generated based on the bounding box of the contour as follows:

- (1) **Grid Cell Generation:** We tile the bounding box of the contour with a uniform grid. Each grid cell has dimensions equal to the maximum macro width $w_{\max}$ and maximum macro height $h_{\max}$ within the cluster, plus half of the minimum channel spacing c on each side, i.e., cell size is $(w_{\max} + c, h_{\max} + c)$. This ensures that when macros are placed at grid positions, they maintain proper spacing.
- (2) **Distance Calculation:** For each grid cell center, we compute its signed Euclidean distance to the nearest point on the contour boundary (using shapely library). A negative distance indicates the cell center is inside the contour; a positive distance indicates it is outside.
- (3) **Grid Selection:** We rank all grid cells by their signed distance (from most negative to most positive) and select the top- k cells, where k equals n , the number of macros in the cluster. This preferentially selects cells inside or near the contour boundary, allowing some flexibility to accommodate macros slightly outside the contour if needed.
- (4) **Feasibility Filtering:** We discard any grid position that would cause a macro to violate die-area boundaries or overlap fixed obstacles.

Note that k and n in the assignment problem formulation refer to the same quantity: the number of macros in the cluster to be legalized.

Macro-to-Grid Assignment. We see legalizing macros to the template grids while minimizing the displacement as an assignment problem on a bipartite graph, where nodes on one side represent all macros in the cluster, and nodes on the other side represent available grid positions in the template. For each macro-grid position pair, we define the edge weight as the Manhattan distance from the macro’s initial position to the grid position. We formulate this as a linear assignment problem:

$$\begin{aligned}
& \min \sum_{m,g} c_{m,g} \cdot x_{m,g} \\
& \text{s.t.} \quad \sum_{j=1}^n x_{ij} = 1, \quad \forall i \in \{1, 2, \dots, n\} \\
& \quad \sum_{i=1}^n x_{ij} = 1, \quad \forall j \in \{1, 2, \dots, n\} \\
& \quad x_{ij} \in \{0, 1\}, \quad \forall i, j \in \{1, 2, \dots, n\}
\end{aligned} \tag{6}$$

where $c_{m,g}$ is the Manhattan distance from the macro’s m initial position to the grid position g , $x_{m,g}$ is the assignment variable. By solving this linear assignment problem, we obtain an optimal macro-to-grid assignment that minimizes total displacement. We employ the Hungarian algorithm to solve it.

The whole flow for template-based macro matching is shown in Fig. 7. Consider the initial macro placement solution generated by mixed-size placement. First, we use the LLM Agent-generated contour algorithm to generate various contours. Subsequently, our designed template generation algorithm produces a template grid based on these contours. Then, the linear assignment problem is solved to assign macros to grids. Finally, multiple macro placement results are generated for each cluster. Here, if only one solution is needed, we can choose the solution with the smallest displacement or the one with the greatest regularity. In our experiments, we chose the solution with the smallest displacement, since it is hard to characterize the trade-off between regularity and displacement. However, the results generated by MacroAgent already improve the regularity of the initial mixed-size placement a lot. Since LLM can efficiently find a lot of heuristics to fit the

real macro legalization testcases, we can obtain an exponential number of solutions for the overall legalization through combination. In the experiment, by sorting the displacements of candidates in each cluster, we quickly identified the global macro legalization candidates (after intra-cluster legalization) with the top K smallest displacements. This **multiple solution capability can increase the robustness of legalization** (find a legal macro legalization).

3.5 Inter-Cluster Refinement

Since each cluster is optimized independently by the LLM to maximize local regularity, the resulting contours may overlap with neighboring clusters. The Inter-Cluster Refinement stage Equation (7) is specifically designed to resolve these global conflicts while strictly preserving the internal relative positions (regularity) achieved by the LLM in each cluster.

Our approach adapts the traditional constraint graph formulation [7] to handle clusters with irregular shapes.

$$\begin{aligned}
\min \quad & \|x - x'\|_1 + \|y - y'\|_1 \\
\text{s.t.} \quad & x_i + w_i \leq x_j, & \forall e_{ij} \in G_x \\
& y_i + h_i \leq y_j, & \forall e_{ij} \in G_y \\
& W_l \leq x_i \leq W_h - w_i \\
& H_l \leq y_i \leq H_h - h_i \\
& x_{a_i^k} - x_{a_j^k} = x'_{a_i^k} - x'_{a_j^k}, \quad \forall a_i^k, a_j^k \in C^k
\end{aligned} \tag{7}$$

As shown in Equation (7), we formulate a linear programming (LP) problem that minimizes the total displacement $\|x - x'\|_1 + \|y - y'\|_1$ between the original positions (x', y') and the legalized positions (x, y) . The first and second constraints enforce non-overlapping conditions through horizontal and vertical constraint graphs G_x and G_y , while the third and fourth constraints ensure that all macros remain within the placement boundaries (W_l for x coordinate of lower left point, W_h for x coordinate of upper right point, H_l for y coordinate of lower left point, H_h for y coordinate of upper right point). The only difference with [7] is the last constraint $x_{a_i^k} - x_{a_j^k} = x'_{a_i^k} - x'_{a_j^k}$ preserves the relative positioning of macros within the same cluster C^k , thus maintaining the regularity.

For scenarios where the linear programming approach fails to solve (due to extra constraints), we employ DREAMPlace’s default heuristic algorithms [17] as a fallback mechanism. Although this approach may compromise the achieved regularity, it provides an efficient solution for resolving overlaps in practice.

4 IMPLEMENTATION DETAILS

The macro legalization algorithm was implemented in Python. This was due to LLMs’ generally better performance with interpreted languages like Python.

We leverage cursor, Cua [40] and pyautogui for constructing the Agent workflow. Cursor is a general AI agent editor, and Cua and pyautogui are used to simulate the mouse and keyboard to control the cursor editor to automate the flow. We use Cua and pyautogui to program the fixed flow. We install the `sequential thinking`, `memory`, and `context7 mcp` in the cursor, endowing the agent with capabilities of thinking, searching and memory. After seeding the template prompt, the agent runs fully automatically: Cursor orchestrates file edits and execution; Cua and pyautogui drive the UI; failure handling (signature mismatch, runtime error, invalid shape) is automated via the corresponding validators. No human edits are applied to candidate code during the search.

We prioritized models that reliably follow structured code-generation instructions and produce long, compositional functions with few syntax errors—key requirements for agentic code generation

(see Section 2.1). In our budgeted setting, Claude 4.0 Sonnet (Thinking, Max) best satisfied these criteria. We also spot-checked Gemini 2.5 Pro and GPT 5 and observed comparable adherence to the prompt scaffolding; a smaller 30-40B-class model struggled with syntax/structure. A formal cross-model benchmark is valuable but outside our scope; we therefore document the budgeted agent setting. The macro legalization source code is publicly available at <https://github.com/gilgamsh/MacroAgent>; the LLM prompts and agent scripts will be released in the same repository upon acceptance to facilitate reproduction and cross-model comparisons.

5 EXPERIMENT

5.1 Experiment Setup

Baseline Selection. DREAMPlace default macro legalization [17] served as the primary baseline method, utilizing heuristic algorithms and constraint graph techniques for macro legalization. Additionally, the sequence pair method presented in [7] is also included; it uses the sequence pair method and simulated annealing to enhance the robustness of macro legalization. Neither of these baseline algorithms explicitly incorporates macro regularity and typically relies on only one or two heuristics.

We distinguish the scope of macro legalization from macro placement. While most prior macro placement works focus on optimizing rough locations that allow minor overlaps [3–9], and rely on a macro legalizer to resolve overlaps, MacroAgent operates as a downstream refinement framework designed to resolve overlaps and improve regularity of existing placements. Therefore, comparing MacroAgent directly with mixed-size global placement-based macro placement tools (which generate the inputs for our system) is methodologically inconsistent. Our work is orthogonal to prior macro placement works and can be integrated with them. Instead, we anchor all methods to the same mixed-size global placement prototype to isolate legalization effects. We validate its performance against state-of-the-art legalization and refinement algorithms (e.g., DREAMPlace’s legalization module and sequence pair method in [7]), ensuring a fair comparison within the same physical design stage. We also compare with the macro legalization capability of Cadence Innovus, a leading industrial physical design tool.

Experiment flow. The experiment workflow starts mixed-size placement using DREAMPlace 4.1 [7]. Subsequently, the macro legalization is applied. Next, we utilize DREAMPlace 4.1 [7] to implement standard cell placement and legalization. Finally, we operate the HeLEM-GR [41] for global routing and evaluate the performance of the resulting layout. We refer to this as the *academic flow*. We additionally evaluate under an *industrial flow*, where standard cell placement and routing are performed entirely within Cadence Innovus (detailed in Section 5.5).

The experiment was configured on a Linux machine running CentOS 7, equipped with an Intel Xeon Platinum 8358 CPU @ 2.60GHz, 1TB of RAM, and an NVIDIA A800-SXM4-80GB GPU (CUDA 12.1). The software environment includes GCC 9.5.0 and Python 3.9.18. The proposed macro legalization algorithm exclusively utilizes CPU resources, while GPU resources are leveraged by DREAMPlace and HeLEM-GR. Cadence Innovus v22.10 is used for the industrial baseline comparison and PPA evaluations. Due to the inherent randomness in sequence pair legalization, we execute the legalization algorithm 20 times with different random seeds. To ensure fairness, MacroAgent generates at most 20 legalization candidate results that minimize displacements. (note that MacroAgent can generate diverse legalization results as needed, **multiple solution capability**) For each result, we perform standard cell placement and global routing. Finally, the result table lists the best wirelength-based outcomes, along with their corresponding legalization runtime, total runtime, and other relevant metrics (total 20 times runtime for sequence pair and MacroAgent).

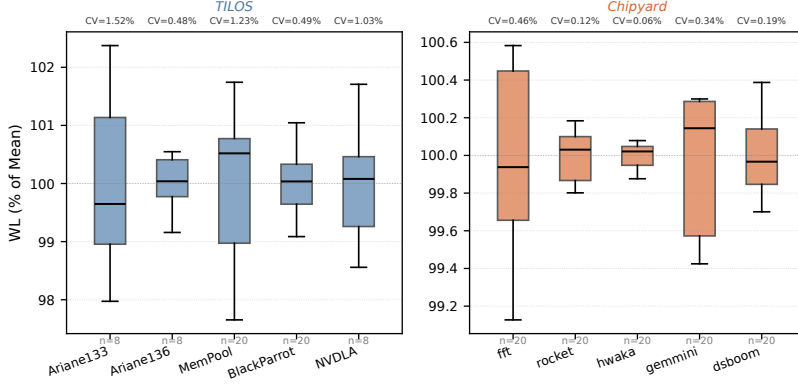


Fig. 8. Normalized wirelength distribution of MacroAgent across all candidate results per testcase.

Table 1. Benchmark statistics of TILOS testcases (easy).

Testcase	#macro	#types	#inst	#net
Ariane133	133	1	98250	101335
Ariane136	136	1	142840	144683
MemPool	20	2	131732	135299
BlackParrot	220	6	1277012	1510338
NVDLA	128	1	153561	208063

To demonstrate the stability of MacroAgent, Fig. 8 shows the distribution of routed wirelength across all candidate results for each testcase, normalized by the per-testcase mean. MacroAgent enumerates candidate solutions in order of increasing displacement, so the resulting variations primarily stem from permutations within small macro clusters, while the placement of large clusters remains stable. Consequently, testcases with diverse macro types (e.g., Chipyard, 7–12 types) exhibit particularly tight distributions ($CV < 0.5\%$), as the dominant clusters are effectively fixed and only minor clusters are permuted. TILOS testcases with a single macro type show slightly higher yet still modest variation ($CV \approx 1\%–1.5\%$), because the single-cluster structure exposes more of the layout to reordering. Overall, the best-of- N selection introduces negligible cherry-picking bias.

Benchmark. For benchmarking, the TILOS [42] benchmark was chosen, following the same testcase configuration as [7]: Ariane133, Ariane136, and MemPool use the ASAP7 PDK, while BlackParrot and NVDLA use the NanGate45 PDK. Furthermore, we found that TILOS is an easy benchmark for macro legalization and that all baseline methods can legalize all testcases. To model more challenging macro legalization scenarios, several macro circuit designs generated via Chipyard [43] (v1.9.1) with the ASAP7 PDK were also incorporated into the benchmarks. The statistics of the TILOS and Chipyard testcases are shown in Table 1 and Table 2. #macro is the number of macros, #types, #inst, and #net denote the number of macro types, the number of instances, and the number of nets. **We can see that the Chipyard testcases are more challenging, having more macros and a wider range of macro types.**

5.2 Prompt Design

To enhance reproducibility, Fig. 9 presents a complete E2 (exploration with synthesis) prompt example used in the contour algorithm discovery. The prompt is organized into the six sections described in Section 3: task description, context with previous results, evolution directive, usage

Table 2. Benchmark statistics of Chipyard testcases (hard).

Testcase	#macro	#types	#inst	#net
fft	121	7	211905	216507
hwaka	292	9	680029	687204
gemmini	737	9	1176503	1189717
dsboom	556	12	643392	659154
rocket	121	7	203995	208595

Table 3. Results on TILOS (academic flow: DREAMPlace + HeLEM-GR): regularity, routed wirelength, congestion (%) and runtime (s, sum of 20 runs). S (Status) shows if failed to legalize.

Testcase	DREAMPlace [17]					Sequence Pair [7]					MacroAgent							
	S	Reg	LT	WL	Con	TT	S	Reg	LT	WL	Con	TT	S	Reg	LT	WL	Con	TT
Ariane133	✓	0.13	0.004	8.69E08	0.03	59.2	✓	0.16	38.625	8.51E08	0.23	376.7	✓	0.67	1.570	7.62E08	0.07	130.9
Ariane136	✓	0.13	0.005	8.37E08	0.02	90.1	✓	0.18	93.652	8.44E08	0.00	576.9	✓	0.59	1.560	8.35E08	0.00	188.4
MemPool	✓	0.13	0.001	7.46E08	0.77	85.0	✓	0.13	1.077	7.44E08	0.48	557.4	✓	0.35	1.050	6.80E08	0.39	501.6
BlackParrot	✓	0.06	0.008	9.10E10	3.90	503.7	✓	0.06	254.212	9.05E10	3.41	3930.7	✓	0.13	5.550	9.19E10	4.28	3850.2
NVDLA	✓	0.08	0.004	2.46E10	4.58	105.5	✓	0.08	87.272	2.42E10	4.12	791.2	✓	0.64	3.470	2.36E10	4.23	329.2
Ratio	1.00	1.00	1.00	1.00	1.00		1.12	16611.63	0.99	2.01	6.93		4.51	663.15	0.95	0.97	4.19	

Table 4. Results on Chipyard (academic flow: DREAMPlace + HeLEM-GR): regularity, routed wirelength, congestion (%) and runtime (s, sum of 20 runs). S (Status) shows if failed to legalize.

Testcase	DREAMPlace [17]						Sequence Pair [7]						MacroAgent					
	S	Reg	LT	WL	Con	TT	S	Reg	LT	WL	Con	TT	S	Reg	LT	WL	Con	TT
fft	×	0.09	0.004	3.16E10	0.33	111.3	×	0.09	2.576	3.16E10	0.34	623.7	✓	0.34	2.740	3.50E10	0.56	656.7
hwaka	×	0.06	0.015	8.40E10	0.29	357.5	✓	0.06	535.677	8.38E10	0.32	2442.0	✓	0.14	6.440	8.50E10	0.29	2199.8
gemmini	✓	0.05	0.029	1.10E11	0.14	354.5	✓	0.05	413.445	1.10E11	0.14	2202.9	✓	0.20	14.630	1.07E11	0.13	2236.5
dsboom	✓	0.05	0.019	7.89E10	0.11	533.8	✓	0.05	409.577	7.89E10	0.12	2404.7	✓	0.15	9.110	7.62E10	0.18	2472.8
rocket	×	0.10	0.006	3.07E10	0.16	862.1	✓	0.10	1.922	3.36E10	0.16	862.1	✓	0.26	2.820	2.87E10	0.11	928.3

scenario, quality metrics, and input references. Slot variables (shown in braces) are automatically populated from the algorithm database at each generation. We choose E2 as the representative example because it is the most comprehensive strategy, requiring the LLM to first summarize common ideas across five parent algorithms and then synthesize a substantially different new heuristic. The highlighted evolution directive is the only section that differs across the five strategies (E1/E2/M1–M3); all other sections remain identical across strategies.

E2 Prompt Example for Contour Algorithm Generation

Task

Design and implement a contour generation algorithm that, given a cluster of macros, produces a closed polygon enclosing them. The algorithm must be implemented as a Python function with the following signature:

```
def generate_contour(corners: list[tuple], centers: list[tuple],
polygons: List[Polygon]) -> Polygon
```

where `corners` is a list of (x,y) coordinates of all macro corners in the cluster, `centers` is a list of (x,y) coordinates of macro centers, `polygons` is a list of Shapely Polygon objects representing each macro's bounding box, and the return value is a Shapely Polygon.

Context

Previous method ideas: {ideas}

Previous results: {displacement}, {regularity_index}, {solutions} and {visualization}

Evolution Directive [E2]:

I have 5 existing contour generation algorithms with their ideas and codes as follows:

No.1 Idea: Compute alpha shape using Delaunay triangulation with adaptive circumradius filtering...

Code: `def generate_contour(corners, centers, ...): ...`

...

No.5 Idea: Grid-based rectilinear boundary tracing with density-aware cell merging...

Code: `def generate_contour(corners, centers, ...): ...`

Please help me create a new contour algorithm that is different from the given ones but can be motivated by them.

Firstly, identify the common idea in the provided algorithms.

Secondly, based on the backbone idea, describe your new algorithm in one sentence.

Thirdly, implement it as a Python function named `generate_contour` following the signature defined above.

Usage Scenario

In the MacroAgent framework, the generated contour serves as the boundary for template generation: the contour polygon is converted into a rectilinear template that defines legal macro slots, and the Hungarian algorithm then assigns macros to these slots to minimize displacement. A contour that closely follows the original macro arrangement preserves macro positions and reduces displacement, while a contour with fewer concavities produces a more regular template with fewer channels. These two objectives often conflict, and the contour algorithm must balance them. The contour algorithm is therefore the key heuristic component whose quality directly determines the downstream legalization result.

Quality Metrics

Regularity: measured by the channel count ratio $R = C_{\min}/C_{\text{actual}}$, where $C_{\min}$ is the theoretical minimum number of channels for a perfect grid arrangement and C_{actual} is the actual channel count computed from the Hanan grid. $R \in (0, 1]$; higher is better ($R = 1$ means a perfect regular array).

Displacement: total Manhattan distance $\sum_i (|x_i - x'_i| + |y_i - y'_i|)$ between each macro's legalized position (x_i, y_i) and its original placement position (x'_i, y'_i) ; lower is better.

Input and Reference

{test case files and reference legalization visualizations}

Fig. 9. Complete E2 (exploration with synthesis) prompt example for contour algorithm generation. The highlighted evolution directive section is the only part that differs across the five strategies (E1/E2/M1–M3); all other sections remain identical. Slot variables (in braces) are populated from the algorithm database at each generation.

5.3 Agent Discovered Contour Algorithms

We use the easy TILOS testcases to generate the contour algorithms, and the hard Chipyard testcases to test the **generalization ability** of the contour algorithm we discovered. Meanwhile, we extract the single-cluster testcases decomposed from TILOS testcases to simplify the problem instance input to the LLM. The LLM-driven algorithm optimization finishes in one day, showing significant efficiency gains versus researchers’ development time. The discovery run is an offline, one-time search. Among the 110 generated candidates, only 8 algorithms remain in the final database because they improve either displacement or regularity on at least one testcase; the rest are discarded during screening. Notably, nearly all candidates are executable and produce valid results. This confirms that our domain-agnostic abstraction effectively reduces the problem to pure geometry reasoning, which is well within the LLM’s capability.

In the initialization stage, the LLM generates $N = 10$ diverse contour heuristics from scratch based solely on the input prompt, without any hand-crafted algorithm design. These heuristics span four broad geometric strategy families: triangulation-based (Items 1, 2, 5), graph-based (Items 4, 10), density/field-based (Items 6, 7, 8), and grid/scanline-based (Items 3, 9):

- (1) **Alpha Shape** – adaptive Delaunay triangulation with circumradius filtering.
- (2) **kNN Concave Hull** – locally adaptive concave boundary via kNN density estimation.
- (3) **Grid-Based** – occupancy grid dilation with convex hull extraction.
- (4) **MST** – buffered minimum spanning tree of macro centers.
- (5) **Corner-Preserving Alpha** – alpha shape augmented with macro corner points.
- (6) **KDE Contour** – Gaussian kernel density iso-contour extraction.
- (7) **Voronoi Clip** – Voronoi cells clipped by local neighborhood radius.
- (8) **Relative Position** – kNN-biased alpha shape preserving local structure.
- (9) **Rectilinear Boundary Tracing** – scanline-based orthogonal interval merging.
- (10) **Manhattan Distance Hull** – L_1 -weighted MST with axis-aligned buffering.

Detailed implementation is available at <https://github.com/gilgamsh/MacroAgent>. This diversity of geometric primitives ensures that the initial population covers a wide region of the algorithm design space, providing a broad foundation for the subsequent evolutionary search.

At test time, each cluster generates 8 candidate legalization results; we can directly find the potential minimum displacement legalization result by combining all clusters’ minimum displacement results. By exploiting the multiple-solution capability, we can find the global macro legalization candidates with the top K smallest displacements. Note that here the displacement is just an estimation, and we still need to perform inter-cluster refinement to get the final legalization result.

As shown in Table 3 and Table 4, MacroAgent outperforms the baselines on both benchmarks (especially on test benchmark Chipyard), showing the generalization ability of the contour algorithms we discovered. **Here, we analyze some of the contour algorithms LLM discovered as shown in Fig. 10.**

Alpha Shape generates the contour as follows: (1) collect corner and center points of all macros; (2) perform Delaunay triangulation on these points; (3) for each triangle, compute its circumradius; (4) discard triangles whose circumradius exceeds an adaptive threshold—a large circumradius indicates an elongated, non-compact triangle that likely spans empty space between macros; (5) merge the remaining compact triangles to form the contour. This yields a tighter, concave boundary as shown in the third figure in Fig. 4.

Grid Based generates the contour as follows: (1) compute the bounding box of all macros and partition it into an $N \times N$ uniform grid; (2) for each grid cell, test whether its center point lies inside any macro polygon and mark it as occupied if so; (3) dilate the occupied cells by marking the eight immediate neighbors of every occupied cell; (4) extract the four corner

```

def create_contour:
    """Alpha Shape"""
    p = centers+corners
    tri = Delaunay(p)
    alpha = median(dist(p))

    for s in tri.simplices
        r = cirradius(p[s])
        if r < alpha
            [loop body]

    return union(tri)

def create_contour:
    """Grid Based"""
    grid = create_grid(bound)

    for rect in rects:
        mark_grid(grid, rect)

    grid = dilate(grid)
    points = corners(grid)
    h = ConvexHull(points)
    ret = h.intersect(rects)

    return ret

def create_contour:
    """MST Concave"""
    p = centers
    G = complete_graph(p)
    for i, j in G.edges():
        d = dist(i, j)
        G[i][j] = d

    mst = mst(G)
    lines = []
    for u, v in mst.edges():
        lines.append(u, v)

    return union(lines)

```

Fig. 10. Selected LLM-designed Contour Algorithms (simplified). Simplified pseudocode of three representative contour generation algorithms discovered by the LLM agent: Alpha Shape (concave boundary via Delaunay triangulation), Grid-Based (occupancy grid with dilation and convex hull), and MST-Based (minimum spanning tree with buffer expansion).

vertices of every marked cell and compute the convex hull; (5) apply light post-processing to obtain the final contour. Since it is based on a grid structure, it produces more regular contours than the other algorithms, as shown in the second figure in Fig. 4.

MST (minimum-spanning tree) generates the contour as follows: (1) construct a complete graph on macro centers with Euclidean edge weights; (2) compute the minimum spanning tree (MST); (3) convert MST edges into line segments and apply a buffer to the segment set; (4) union the buffered region with all macro polygons to obtain the final contour. This method naturally fits the outline of the original macros, with only a simple effect of smoothing the boundaries, as shown in the last figure in Fig. 4.

From the above analysis, we can see that the LLM designed contour algorithms can achieve different trade-offs between regularity and displacement in algorithm level, not only in parameter level. **These algorithms are meaningful and readable.** We also find that with the help of reference manual legalization solutions, LLMs can generate more focused thoughts and modifications.

Quantitative Diversity Analysis. To quantitatively evaluate the diversity of the eight LLM-discovered contour algorithms, we compute the pairwise Intersection-over-Union (IoU) of the contour polygons generated by each algorithm on a representative testcase (Ariane133). A lower IoU between two algorithms indicates that they produce geometrically distinct contours, confirming genuine algorithmic diversity rather than minor parametric variations. Fig. 11 reports the full 8×8 pairwise IoU matrix. The average off-diagonal IoU is 0.71. Because all contours enclose the same set of macros, a baseline overlap is inherent; the meaningful variation lies in boundary strategies, where the algorithms differ significantly. Notably, *kde* exhibits the lowest average pairwise IoU with all other methods (0.48–0.77), confirming that it explores a fundamentally different geometric strategy. While some method pairs share higher similarity (e.g., *mst* and *density_weighted* at 0.94; *alpha_shape* and *mst* at 0.89), the eight algorithms span a wide range of contour shapes, which directly enables the multiple-solution capability described above.

5.4 Overall Comparisons

Regularity reduces channel count and deadspace, which in turn reduces detours and router effort. We therefore report routed wirelength and congestion as PPA-relevant surrogates in our legalization only study. We list the results of the TILOS and Chipyard testcases in Table 3 and Table 4. S shows whether the macro legalization is successful. Reg shows the regularity score of the macro

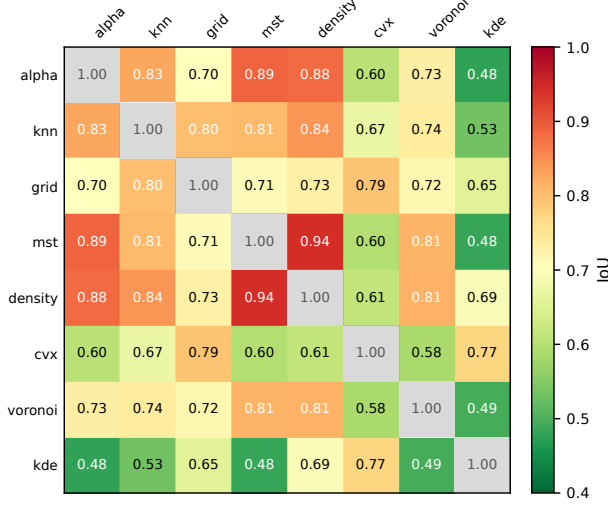


Fig. 11. Pairwise IoU matrix of eight contour algorithms on Arianel33. Lower off-diagonal values indicate higher geometric diversity.

legalization. LT shows the time of macro legalization. WL shows the routed wirelength reported by HeLEM-GR [41] in DREAMPlace internal units. Con shows the congestion(%). Regularity (Reg) is calculated using the channel-based metric defined in Equation (2), where a score of 1.0 represents a theoretically perfect square array. TT shows the total workflow time(s) (sum of 20 runs), including mixed-size placement, macro legalization, standard cell placement and legalization, and global routing.

For the TILOS benchmark, MacroAgent achieves an average 4% improvement in wirelength and reduces congestion in some testcases compared with sequence pair. Compared with DREAMPlace default macro legalization, MacroAgent delivers a 5% improvement in wirelength and similar congestion. Beyond standard metrics, the robustness of MacroAgent significantly surpasses other methods, particularly in complex design scenarios. As evidenced in Table 4, the DREAMPlace baseline fails to legalize three out of five Chipyard testcases (‘fft’, ‘hwaka’, ‘rocket’), and the Sequence Pair method fails on ‘fft’ despite 20 restart attempts. These failures typically stem from the limitation of utilizing a single heuristic, which often leads to getting trapped in local optima or failing to resolve dense overlaps in highly constrained regions. In contrast, MacroAgent leverages a portfolio of diverse LLM-designed heuristics. By generating multiple candidate contours and templates for each cluster, our framework effectively avoids the rigidity of single-algorithm approaches. This capability ensures design closure even in high-density cases.

Furthermore, as shown in Table 5, MacroAgent achieves a wirelength improvement of 3% to 5% with similar congestion compared to both DREAMPlace and sequence pair on the common success testcases. Note that for failed cases (marked with × in Table 4), the reported wirelength and congestion values are obtained by proceeding with the downstream flow despite remaining macro overlaps; these results are not physically valid and are included only for reference—they should not be used for quantitative comparison.

The legalization efficiency of MacroAgent is higher than sequence pair (since sequence pair use simulated annealing), with a runtime on the order of seconds and its time consumption in the full workflow being negligible. **Overall our MacroAgent framework is more robust, effective with better solution quality.**

Table 5. Metric ratio comparison of MacroAgent and baselines on Chipyard.

Comparison	Reg	LT	WL	Con	TT
MacroAgent / DREAMPlace	3.50	491.98	0.97	1.28	5.47
MacroAgent / Sequence Pair	2.98	0.38	0.95	1.01	1.01

Table 6. Comparison of post-route PPA results on TILOS benchmark (industrial flow: Innovus place-and-route): frequency (MHz), routed wirelength (μm), WNS (ns), TNS (ns), and power (mW).

Testcase	Freq (MHz)	DREAMPlace [17]				Sequence Pair [7]				MacroAgent			
		WL (μm)	WNS (ns)	TNS (ns)	Power (mW)	WL (μm)	WNS (ns)	TNS (ns)	Power (mW)	WL (μm)	WNS (ns)	TNS (ns)	Power (mW)
Ariane133	500	910082	-0.128	-2.876	235.638	907235	-0.115	-2.533	235.551	883726	-0.066	-0.882	234.954
Ariane136	769	906339	-0.021	-1.554	367.128	910466	-0.028	-1.822	367.467	857138	-0.007	-0.060	366.547
MemPool	333	778501	-0.396	-1207.310	55.674	775212	-0.395	-1237.757	55.599	752503	-0.369	-1048.01	55.613
BlackParrot	667	30538061	0.144	0	2072.170	30395222	0.150	0	2071.562	29831309	0.146	0	2071.842
NVDLA	500	8850019	-0.208	-2.355	1093.415	8852443	-0.222	-3.275	1093.389	8806758	-0.021	-0.129	1092.430
Ratio	-	1.000	1.000	1.000	1.000	0.998	1.050	1.117	0.999	0.971	0.574	0.317	0.998

5.5 PPA Evaluation

To validate the downstream impact of macro legalization on physical design quality, we perform complete place-and-route using Cadence Innovus on the TILOS benchmark, where all baseline methods successfully legalize every testcase, enabling a fair PPA comparison. On Chipyard, baseline failures preclude meaningful PPA comparison.

Experiment Flow. After macro legalization, we import the post-legalization DEF and LEF files into Innovus and run `place_opt_design` for standard cell placement and optimization, followed by routing. We report routed wirelength, WNS (worst negative slack), TNS (total negative slack), and total power. For each testcase, we set a target frequency (i.e., the reciprocal of the clock period) so that WNS falls within 10–20% of the clock period. A negative WNS means the critical path violates the timing constraint; a more negative value indicates a more severe violation. A positive WNS indicates that all paths meet timing, with a larger value reflecting more slack margin. In both cases, a higher WNS is preferable. TNS aggregates all negative slacks across timing endpoints (0 if no negative slack), reflecting the overall timing condition of the design.

Results Analysis. As shown in Table 6, MacroAgent provides the strongest overall PPA trade-off among the three methods. Compared with DREAMPlace, MacroAgent reduces routed wirelength by 2.9% on average, with the largest improvement on Ariane136 (−5.4%). The improved regularity reduces deadspace between macros, which shortens routing detours and lowers wirelength; the wirelength reduction in turn improves downstream PPA. This wirelength reduction directly translates into timing improvements: MacroAgent achieves 42.6% WNS improvement and 68.3% TNS improvement on average compared with DREAMPlace. The gains are particularly notable on Ariane136 and NVDLA, where TNS is reduced by over 90%. Power consumption remains comparable across all methods, as macro legalization primarily affects interconnect topology rather than cell-level switching activity. MacroAgent achieves lower routed wirelength on all five testcases and better overall timing than Sequence Pair (best WNS on four of five testcases and equal-best TNS on all five), with comparable power.

Table 7. Comparison with Innovus macro placement on TILOS (industrial flow: Innovus place-and-route): routed wirelength (μm), congestion (%), and runtime (s).

Testcase	Innovus			MacroAgent		
	WL (μm)	Con (%)	RT (s)	WL (μm)	Con (%)	RT (s)
Ariane133	727027	0.02	408	701629	0.01	379
Ariane136	893684	0.01	491	892388	0.01	437
MemPool	821082	0.01	432	810131	0.01	417
BlackParrot	28421983	0.16	2938	28341650	0.29	2635
NVDLA	8325486	0.63	551	8033640	0.61	512
Ratio	1.000	1.00	1.00	0.982	1.06	0.92

Comparison with Innovus Macro Placement. To further evaluate MacroAgent against industrial tools, we construct two controlled flows on the five TILOS testcases. **Flow A (Innovus baseline):** DREAMPlace initial placement $\rightarrow$ Innovus macro refinement via `place_design -concurrent_macros -incremental` followed by `refine_macro_place` (which includes flip optimization) $\rightarrow$ `place_design` for standard cell placement $\rightarrow$ routing. **Flow B (MacroAgent + Innovus):** DREAMPlace initial placement $\rightarrow$ MacroAgent macro legalization $\rightarrow$ the same `refine_macro_place` and downstream stages as Flow A. Both flows share identical Innovus settings and the same downstream pipeline after macro refinement. Because flip optimization cannot be disabled in `refine_macro_place` and significantly affects wirelength, both flows include it. Therefore, the only controlled variable is the legalized macro placement entering the shared Innovus pipeline, isolating the impact of macro legalization quality on final PPA.

As shown in Table 7, MacroAgent achieves lower routed wirelength than Innovus on all five testcases, with an average improvement of 1.8%. The improvement is most pronounced on Ariane133 and NVDLA (both -3.5%). Since Innovus’s macro placement refinement includes flip optimization, it may further adjust macro positions based on orientation changes—an optimization that can benefit certain testcases. This explains the congestion advantage of Innovus on BlackParrot (0.16% vs. 0.29%): Innovus’s integrated orientation-flip refinement effectively reduces routing hotspots. However, flip optimization does not always help: compared to the results without Innovus refinement in Table 6, the wirelength of Ariane136 and MemPool increases in both flows, suggesting that the orientation changes can degrade placement quality on certain designs. Despite this, MacroAgent still achieves consistently lower wirelength across all testcases, indicating that its legalization provides a superior starting point that Innovus’s downstream optimization preserves and benefits from.

5.6 Analysis and Visualization

To intuitively understand the impact of our MacroAgent legalization, we choose `rocket` to visualize the results of each legalization method. The visualization results of `rocket` are shown in Fig. 12. Initial shows the layout after mixed-size placement, which is the input of macro legalization. We can see that the layout after DREAMPlace macro legalization still has overlaps, which shows that one or two heuristics cannot cover all testcases. Although sequence pair can resolve the overlaps, the regularity is not satisfactory, resulting in irregular channels, which is harmful to wirelength. On the last figure, MacroAgent shows that the layout resolves the overlaps, and the regularity is significantly improved.

To further demonstrate the effectiveness of LLM-generated multiple contour algorithms, we present the wirelength ratio (same configuration with the main experiment) on the TILOS Benchmark

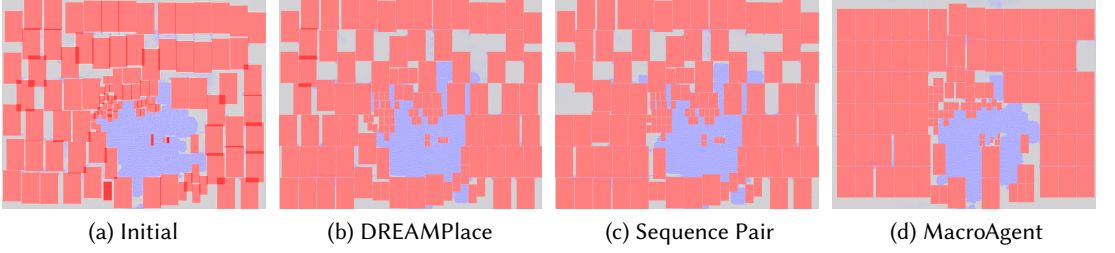


Fig. 12. Comparison of different macro legalization methods on `rocket`: (a) initial global placement, (b) DREAMPlace legalization with remaining overlaps, (c) Sequence Pair legalization with irregular channels, and (d) MacroAgent legalization with overlap-free and regular layout.

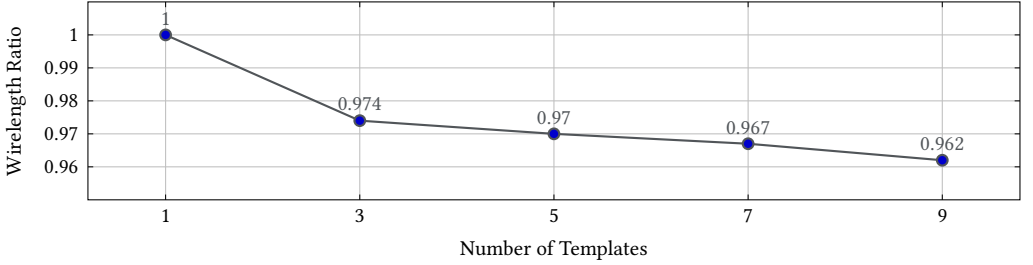


Fig. 13. The impact of the used contour algorithm number on the average routed wirelength ratio in the TILOS benchmark (starts with only a rectangle contour).

under two scenarios: simply using the rectangle contour algorithm and continuously adding LLM-generated algorithms. The x-axis counts the number of distinct LLM-designed contour algorithms included in the candidate set (starting from the rectangle baseline = 1). Each contour induces one template; we then pick the best legalization per cluster via assignment. We can see that wirelength is improved when multiple LLM-generated algorithms are used, as shown in Fig. 13, which shows that LLM-generated algorithms are better and multiple heuristics can help improve the overall performance.

5.7 Discussions

Previous work [32] indicate that LLMs exhibit limitations in both domain-specific EDA knowledge and large-scale project handling capabilities, resulting in suboptimal direct code generation. To address these challenges, we developed a framework that abstracts macro legalization into a geometric problem. Notably, when generating algorithms for this abstracted problem, LLMs successfully leveraged universal geometric concepts such as alpha shapes, k-nearest neighbors, and minimum spanning trees, rather than relying on EDA-specific domain knowledge. Importantly, our framework employs a general-purpose, off-the-shelf LLM without any domain-specific fine-tuning. Domain-specific decisions—such as the clustering strategy and the choice of optimization objectives (regularity and displacement minimization)—are encoded by human engineers in the framework design, while the LLM operates solely on the abstracted geometric subproblem using its pretrained reasoning capabilities. This domain-agnostic approach yielded promising results.

A traditional automation loop optimizes parameters within a fixed algorithm. In contrast, the LLM Agent demonstrated the ability to traverse the algorithmic search space, proposing distinct geometric heuristics (e.g., switching from convex hulls to MST-based contours) that a parameter-sweep baseline could not discover.

Our algorithm enforces regularity solely within intra-cluster legalization, while the fallback heuristic algorithms for inter-cluster legalization compromise layout regularity. Meanwhile, the current LLM Agent-based algorithm design approach remains limited to generating contour algorithms. Future research should prioritize advanced regularity-aware heuristic methodologies for clustering and inter-cluster optimization.

6 CONCLUSION

We introduce MacroAgent, a robust regularity-aware macro legalization framework utilizing LLM-agent-designed contour algorithms. The framework proceeds through four stages: clustering, regularity-aware contour generation, template-based matching, and inter-cluster refinement. We abstract the EDA problem of macro legalization into a domain-agnostic geometric problem, enabling LLMs to design diverse efficient heuristic contour algorithms for macro legalization. On TILOS benchmarks, MacroAgent reduces wirelength by 5% compared to DREAMPlace 2.0 [17] and 4% compared to the sequence pair [7], while matching their congestion qualities. On Chipyard designs, it successfully legalizes every case and achieves wirelength improvements of 3% to 5%. End-to-end evaluation through Cadence Innovus further confirms that the regularity improvements yield tangible PPA gains, including 2.9% lower routed wirelength and 68.3% TNS improvement over the DREAMPlace macro legalization baseline, and 1.8% lower routed wirelength when integrated into the Innovus macro placement flow. Our approach shortens development cycles and highlights the potential of LLMs to assist in designing EDA algorithms. We believe this paradigm can be extended to other NP-hard EDA algorithm design problems, bringing new vitality to the community.

REFERENCES

- [1] P. McLellan, “Innovus mixed placer,” https://community.cadence.com/cadence_blogs_8/b/breakfast-bytes/posts/innovus-mixed-placer, October 6 2020, cadence Blogs - Breakfast Bytes, accessed 14 September 2025.
- [2] X. Gao, Y.-M. Jiang, L. Shao, P. Raspopovic, M. E. Verbeek, M. Sharma, V. Rashingkar, and A. Jalota, “Congestion and timing aware macro placement using machine learning predictions from different data sources: Cross-design model applicability and the discerning ensemble,” in *Proceedings of the 2022 International Symposium on Physical Design*, 2022, pp. 195–202.
- [3] Y. Pu, T. Chen, Z. He, C. Bai, H. Zheng, Y. Lin, and B. Yu, “Incremacro: Incremental macro placement refinement,” in *Proceedings of the 2024 International Symposium on Physical Design*, 2024, pp. 169–176.
- [4] J.-M. Lin, Y.-L. Deng, S.-T. Li, B.-H. Yu, L.-Y. Chang, and T.-W. Peng, “Regularity-aware routability-driven macro placement methodology for mixed-size circuits with obstacles,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 27, no. 1, pp. 57–68, 2018.
- [5] A. Agnesina, P. Rajvanshi, T. Yang, G. Pradipta, A. Jiao, B. Keller, B. Khailany, and H. Ren, “Autodmp: Automated dreamplace-based macro placement,” in *Proceedings of the 2023 International Symposium on Physical Design*, 2023, pp. 149–157.
- [6] K. Peng and W. Zhu, “Pplace-ms: Methodologically faster poisson’s equation-based mixed-size global placement,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 2, pp. 613–626, 2023.
- [7] Y. Chen, Z. Wen, Y. Liang, and Y. Lin, “Stronger mixed-size placement backbone considering second-order information,” in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2023, pp. 1–9.
- [8] K. Xue, R.-T. Chen, X. Lin, Y. Shi, S. Kai, S. Xu, and C. Qian, “Reinforcement learning policy as macro regulator rather than macro placer,” in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- [9] J. Jiang, Y. Pu, M. Li, Y. Zhao, P. Liao, Z. Zhang, Y. Lin, and B. Yu, “Regplace: Regularity-aware placement for full-system dnn accelerator designs,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2025.
- [10] T.-C. Chen, P.-H. Yuh, Y.-W. Chang, F.-J. Huang, and D. Liu, “Mp-trees: A packing-based macro placement algorithm for mixed-size designs,” in *Proceedings of the 44th annual Design Automation Conference*, 2007, pp. 447–452.
- [11] C.-H. Chang, Y.-W. Chang, and T.-C. Chen, “A novel damped-wave framework for macro placement,” in *2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2017, pp. 504–511.
- [12] A. Vidal-Obiols, J. Cortadella, J. Petit, M. Galceran-Oms, and F. Martorell, “Rtl-aware dataflow-driven macro placement,” in *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2019, pp. 186–191.
- [13] A. B. Kahng, R. Varadarajan, and Z. Wang, “Rtl-mp: toward practical, human-quality chip planning and macro placement,” in *Proceedings of the 2022 International Symposium on Physical Design*, 2022, pp. 3–11.

- [14] —, “Hier-rtlmp: A hierarchical automatic macro placer for large-scale complex ip blocks,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 5, pp. 1552–1565, 2023.
- [15] *Innovus User Guide*, 23rd ed., Cadence Design Systems.
- [16] J. Cong and M. Xie, “A robust mixed-size legalization and detailed placement algorithm,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 27, no. 8, pp. 1349–1362, 2008.
- [17] Y. Lin, D. Z. Pan, H. Ren, and B. Khailany, “Dreamplace 2.0: Open-source gpu-accelerated global and detailed placement for large-scale vlsi designs,” in *2020 China Semiconductor Technology International Conference (CSTIC)*. IEEE, 2020, pp. 1–4.
- [18] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altschmidt, S. Altman, S. Anadkat *et al.*, “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [19] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi *et al.*, “Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning,” *arXiv preprint arXiv:2501.12948*, 2025.
- [20] H. Wang, T. Fu, Y. Du, W. Gao, K. Huang, Z. Liu, P. Chandak, S. Liu, P. Van Katwyk, A. Deac *et al.*, “Scientific discovery in the age of artificial intelligence,” *Nature*, vol. 620, no. 7972, pp. 47–60, 2023.
- [21] J. Ahn, R. Verma, R. Lou, D. Liu, R. Zhang, and W. Yin, “Large language models for mathematical reasoning: Progresses and challenges,” in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: Student Research Workshop*, 2024, pp. 225–237.
- [22] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin *et al.*, “Metagpt: Meta programming for a multi-agent collaborative framework,” in *The Twelfth International Conference on Learning Representations*.
- [23] B. Romera-Paredes, M. Barekatin, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi *et al.*, “Mathematical discoveries from program search with large language models,” *Nature*, vol. 625, no. 7995, pp. 468–475, 2024.
- [24] M. Liu *et al.*, “Verilogval: Evaluating large language models for verilog code generation,” in *ICCAD*. IEEE, 2023.
- [25] K. Chang *et al.*, “Data is all you need: Finetuning llms for chip design via an automated design-data augmentation framework,” in *DAC*, 2024.
- [26] Y. Fu *et al.*, “Gpt4aigchip: Towards next-generation ai accelerator design automation via large language models,” in *ICCAD*. IEEE, 2023.
- [27] W. Banzhaf, F. D. Francone, R. E. Keller, and P. Nordin, *Genetic programming: an introduction: on the automatic evolution of computer programs and its applications*. Morgan Kaufmann Publishers Inc., 1998.
- [28] M. O’Neill, L. Vanneschi, S. Gustafson, and W. Banzhaf, “Open issues in genetic programming,” *Genetic Programming and Evolvable Machines*, vol. 11, no. 3, pp. 339–363, 2010.
- [29] A. Sypula, P. Yin, J. Lacomis, C. Le Goues, E. J. Schwartz, and G. Neubig, “Learning to superoptimize real-world programs,” in *Deep Learning for Code Workshop (ICLR 2022 Workshop)*, 2022.
- [30] D. J. Mankowitz, A. Michi, A. Zhernov, M. Gelmi, M. Selvi, C. Paduraru, E. Leurent, S. Iqbal, J.-B. Lespiau, A. Ahern *et al.*, “Faster sorting algorithms discovered using deep reinforcement learning,” *Nature*, vol. 618, no. 7964, pp. 257–263, 2023.
- [31] F. Liu, T. Xialiang, M. Yuan, X. Lin, F. Luo, Z. Wang, Z. Lu, and Q. Zhang, “Evolution of heuristics: Towards efficient automatic algorithm design using large language model,” in *Forty-first International Conference on Machine Learning*.
- [32] F. Liu *et al.*, “A systematic survey on large language models for algorithm design,” *arXiv*, 2024.
- [33] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, “Large language models as optimizers,” in *The Twelfth International Conference on Learning Representations*.
- [34] G. Jawahar, M. Abdul-Mageed, L. Lakshmanan, and D. Ding, “Llm performance predictors are good initializers for architecture search,” in *Findings of the Association for Computational Linguistics ACL 2024*, 2024, pp. 10 540–10 560.
- [35] X. Yao, J. Jiang, Y. Zhao, P. Liao, Y. Lin, and B. Yu, “Evoplace: Evolution of optimization algorithms for global placement via large language models,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2026.
- [36] A. Novikov, N. Vü, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. Ruiz, A. Mehra-bian *et al.*, “Alphaevolve: A coding agent for scientific and algorithmic discovery,” *arXiv preprint arXiv:2506.13131*, 2025.
- [37] C. Yu, R. Liang, C.-T. Ho, and H. Ren, “Autonomous code evolution meets np-completeness,” *arXiv preprint arXiv:2509.07367*, 2025.
- [38] M. Andrews and S. Witteveen, “Gpu kernel scientist: An llm-driven framework for iterative kernel optimization,” *arXiv preprint arXiv:2506.20807*, 2025.
- [39] S. Liu, J. Jiang, Z. He, Z. Wang, Y. Lin, B. Yu, and M. Wong, “Routing-aware legal hybrid bonding terminal assignment for 3d face-to-face stacked ics,” in *Proceedings of the 2024 International Symposium on Physical Design*, 2024, pp. 75–82.
- [40] “c/ua is the Docker Container for Computer-Use AI Agents,” <https://github.com/trycua/cua>.

- [41] C. Zhao, Z. Guo, R. Wang, Z. Wen, Y. Liang, and Y. Lin, “Helem-gr: Heterogeneous global routing with linearized exponential multiplier method,” in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, 2024, pp. 1–9.
- [42] C.-K. Cheng, A. B. Kahng, S. Kundu, Y. Wang, and Z. Wang, “Assessment of reinforcement learning for macro placement,” in *Proceedings of the 2023 International Symposium on Physical Design*, 2023, pp. 158–166.
- [43] A. Amid, D. Biancolin, A. Gonzalez, D. Grubb, S. Karandikar, H. Liew, A. Magyar, H. Mao, A. Ou, N. Pemberton, P. Rigge, C. Schmidt, J. Wright, J. Zhao, Y. S. Shao, K. Asanović, and B. Nikolić, “Chipyard: Integrated design, simulation, and implementation framework for custom socs,” *IEEE Micro*, vol. 40, no. 4, pp. 10–21, 2020.